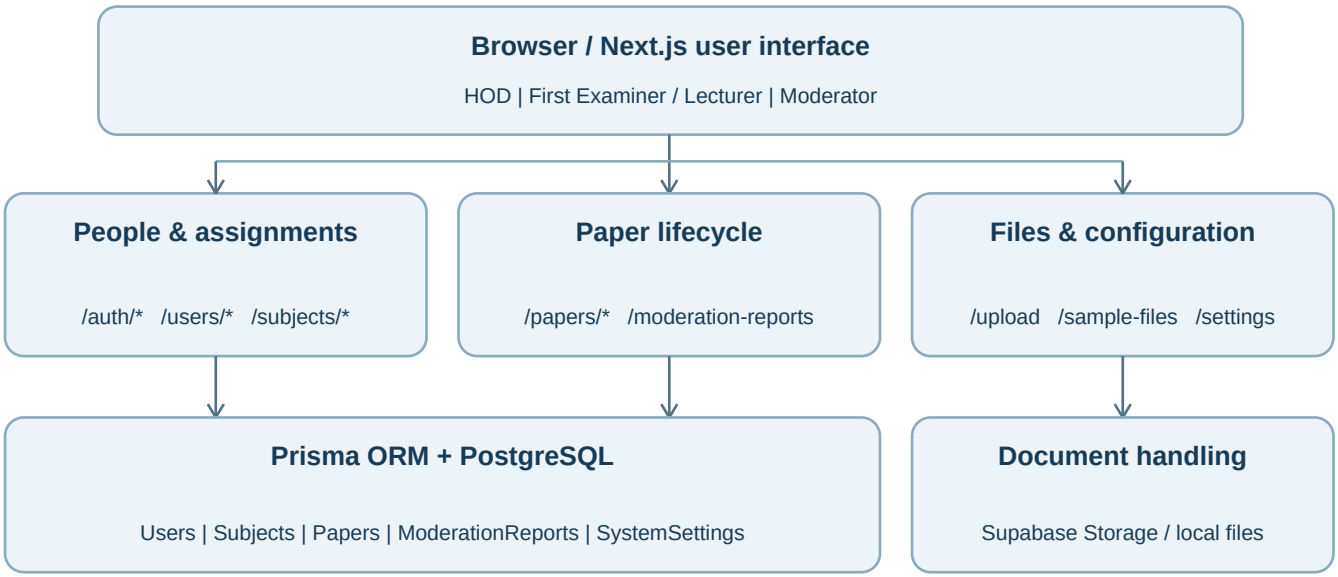
Examination Paper Moderation System

Architecture and API overview | Source-based presentation reference

API surface	Transport	Implementation
13 paths / 24 operations	Same-origin HTTP; JSON + multipart + binary files	Next.js App Router route handlers



Download route: resolve document -> verify password -> encrypt or return unlocked bytes.

How to read this map

Paths are relative to the deployed application origin. Example: **https://your-app.example/api/subjects**. Replace **{id}** with a real database identifier. Role labels describe intended UI workflows, not proof of server-side authentication.

Scope & provenance

Reviewed all 13 route.ts files under src/app/api at application revision c55687a (merged PR #3). Documentation-only changes; no API behavior was modified. This map describes the current code, not a live deployment test.

Accounts & assignments

Endpoint reference | 12 operations

Method / path	Request / purpose	Success response
POST /api/auth/login	JSON: email, password, department. Validate credentials and department.	200 mapped user; no session token issued.
POST /api/auth/register	JSON: name, email, password, department; optional confirmPassword, contact.	201 mapped user; Faculty Board approval false.
GET /api/users	Optional query: department. List staff.	200 user array.
POST /api/users	JSON: name, email, password; optional department, isDepartmentHead, contact, facultyBoardApproved.	201 created user.
PUT /api/users/{id}	JSON: name, email, role, department, contact, facultyBoardApproved (provided fields updated).	200 updated user.
PATCH /api/users/{id}	JSON: currentPassword, newPassword. Change password.	200 {success: true}.
DELETE /api/users/{id}	Delete user and associated papers/reports; clean subject assignments.	200 {success: true}.
GET /api/subjects	Optional query: department. List subjects and assignments.	200 enriched subject array with papers.
POST /api/subjects	JSON: code, name, semester; optional academicYear, department, firstExaminerId, lecturerId, lecturerIds, moderatorId, assignedByUserId.	201 created subject.
GET /api/subjects/{id}	Path: subject ID. Read subject and assigned staff.	200 enriched subject; 404 if absent.
PUT /api/subjects/{id}	JSON: code, name, academicYear, semester and assignment fields; optional assignedByUserId.	200 updated subject; draft owner synchronized if First Examiner changes.
DELETE /api/subjects/{id}	Delete subject and associated papers/reports.	200 {success: true}.

Assignment rules: First Examiner and Moderator cannot be the same person. Faculty Board approval is checked for submitted staff assignments. HOD verification runs only when assignedByUserId is supplied.

Papers & moderation

Endpoint reference | 6 operations

Method / path	Request / purpose	Success response
GET /api/papers	Query: lecturerId, moderatorId, department (optional). Lecturer filter takes precedence if both IDs are given.	200 mapped paper array; may include virtual subject placeholders.
POST /api/papers	JSON: subjectId, lecturerId; optional academicYear, semester, status and document URL fields.	201 created paper. Supplied lecturerId checked against assigned setter, when assigned.
GET /api/papers/{id}	Read paper. Also accepts virtual-{subjectId} for a subject without a saved paper.	200 mapped paper or virtual draft; 404 if missing.
PUT /api/papers/{id}	JSON: status, document URL fields, filePassword, actingUserId (all optional).	200 updated paper. Finalization marks sentToHod and records timestamp.
GET /api/moderation-reports	Query: paperId, moderatorId, department (optional). paperId selects a single report and bypasses other filters.	200 report object with paperId; otherwise report array. 404 when requested report is missing.
POST /api/moderation-reports	JSON: paperId, moderatorId, remarks required; optional additionalComments, followUpActions, signatures, report URLs, preserveStatus.	201 upserted report. Sets REVISION_REQUIRED unless preserveStatus is truthy.

Document fields

paperUrl, paperWordUrl, markingSchemeUrl, markingSchemeWordUrl hold the primary file locations. Draft URL fields are preserved or updated during moderation/finalization. Reports can store reportPdfUrl, reportWordUrl, moderatorSignatureUrl and lecturerSignatureUrl.

Paper state map

Typical UI stage	Database state	API action
Create draft	DRAFT	POST /api/papers
Submit / resubmit for review	UNDER_MODERATION	PUT /api/papers/{id} with status under_moderation
Moderator requests revision	REVISION_REQUIRED	POST /api/moderation-reports (default behavior)
Finalize and send to HOD	FINALIZED	PUT /api/papers/{id} with status finalized

This is the intended workflow, not an enforced transition graph. The API also maps submitted to UNDER_MODERATION and unknown status strings to DRAFT. First Examiner checks on PUT run only when actingUserId is supplied and the resulting state is UNDER_MODERATION or FINALIZED.

Files & configuration

Endpoint reference | 6 operations

Method / path	Request / purpose	Success response
POST /api/upload	multipart/form-data: file required; folder optional (default files). Accepts PDF, DOC/DOCX and supported signature images.	200 {url, path}. Supabase uploads bucket if configured; otherwise public/uploads.
GET /api/sample-files	Query: code; kind=paper|scheme; format=pdf|docx; version=draft|final. Defaults: CMIS 3134, paper, pdf, draft.	200 generated binary PDF/DOCX. PDF inline; DOCX attachment.
GET /api/papers/{id}/download	Query: userId. Checks same-department HOD, assigned Moderator or First Examiner.	200 {authorized, filePassword, hasPaperPdf, hasPaperWord, hasSchemePdf, hasSchemeWord}.
POST /api/papers/{id}/download	JSON: userId, password, fileType; optional mode=encrypted|unlocked. fileType: paperPdf, paperWord, schemePdf, schemeWord.	200 binary PDF/DOCX. Encrypted attachment by default; unlocked inline if requested.
GET /api/settings	Read academicYear and semester. Defaults used if database access fails.	200 settings object.
PUT /api/settings	JSON key/value object; values converted to strings and upserted.	200 {success:true}; may include warning if persistence fails.

Presentation sequence

1. Prepare - HOD uses users and subjects APIs to approve staff and assign examiners.

2. Set & submit - First Examiner uploads files, saves returned URLs to the paper, and submits it for moderation.

3. Review & revise - Moderator reads assigned papers and saves a report; First Examiner revises and resubmits.

4. Finalize & retrieve - First Examiner finalizes with a password. HOD/Moderator use the download route to verify the password and retrieve document bytes.

Download details: Source bytes may come from generated samples, local uploads, or an HTTP(S) URL. Missing document slots fall back to generated sample documents. The download handler does not itself require the paper state to be FINALIZED.

Contracts & implementation notes

Presenter notes | Separate current behavior from production recommendations

Representative requests

Call	Illustrative body / query
POST /api/auth/login	{"email":"staff@example.edu", "password":"<password>", "department":"<department>"}
PUT /api/papers/{id}	{"status":"finalized", "actingUserId":"<first-examiner-id>", "filePassword":"<chosen-password>"}
POST /api/papers/{id}/download	{"userId":"<staff-id>", "password":"<file-password>", "fileType":"paperPdf", "mode":"encrypted"}

Response conventions

JSON is the default request/response format except multipart uploads and binary document responses. Common error shape: **{"error":"message"}**. Route-dependent statuses: 400 validation, 401 invalid login/file password, 403 access/actor rejection, 404 missing resource/report and 500 exceptions. Creation endpoints normally return 201; upload and other successful reads/updates return 200. Settings handlers can return 200 fallbacks/warnings instead of an error.

Important current security limitations

• Login returns a user object; AuthContext persists it in browser localStorage. The reviewed API handlers do not establish or validate a server session/JWT.

• Several routes have no server-side role check. Some checks trust optional request IDs (assignedByUserId, actingUserId); download access trusts the supplied userId. These are not authenticated identities.

• Passwords are compared/stored as plaintext in the reviewed code. Some raw Prisma responses and nested relations can expose password fields. GET download explicitly returns the file protection password.

• Uploads can return public URLs; generated samples and source file URLs are separate access paths from the encrypted download endpoint. Encryption on download is not equivalent to protecting stored source documents.

Before production

Use server-authenticated sessions and mandatory role/ownership checks; hash account passwords; remove sensitive fields from responses; use private document storage and controlled retrieval; validate payloads/file sizes and remote download sources; review rate limiting and audit logging. These are recommendations, not claims of completed work.

Source index

src/app/api/{auth/login, auth/register, users, users/[id], subjects, subjects/[id], papers, papers/[id], papers/[id]/download, moderation-reports, upload, sample-files, settings}/route.ts; src/lib/AuthContext.tsx; src/lib/prisma.ts.